Name: **Muhammad Umair Shaukat**

Sap ID: **58954**

Section: **BSCS-5A**

Course: **Game Programming**

Assignment 3

Multiplayer Game Code Architecture Design

League of Legends (MOBA)

1. Introduction

League of Legends, commonly referred to as LoL, is one of the most widely played online multiplayer games in the world. Developed and published by Riot Games, it falls under the Multiplayer Online Battle Arena (MOBA) genre, a category defined by large-scale team-based competition in a shared virtual environment. Since its release in 2009, the game has attracted tens of millions of active players globally and continues to be a dominant title in the esports industry.

From a software engineering perspective, League of Legends represents an extraordinarily complex system. It is a real-time, competitive multiplayer game in which ten players simultaneously interact with a shared game world over a network connection. Each player controls a unique character, and the game must track hundreds of simultaneous events — from champion movements to projectile trajectories and spell effects — all while maintaining a consistent and synchronized game state across all connected clients.

The importance of software architecture in a game of this scale cannot be overstated. Without a well-planned architectural foundation, the codebase would quickly become unmanageable, prone to bugs, and difficult to extend. A solid architecture ensures that different game systems, such as combat, networking, AI, and UI, remain loosely coupled and independently testable. It also supports the addition of new champions, maps, and features without requiring significant rework of existing systems.

The purpose of this report is to analyze and propose a software architecture for recreating a simplified version of League of Legends using the Unity game engine. This includes examining all major game systems, defining class structures, explaining how these systems communicate, and justifying the design decisions made throughout the process. The approach taken is grounded in object-oriented programming principles and Unity's component-based development model.

2. Game Overview

League of Legends is a team-based strategy game played on a fixed map called Summoner's Rift. Two teams of five players each compete to destroy the opposing team's Nexus — the core structure located in their base. The match takes place in real time and typically lasts between twenty and fifty minutes, depending on the strategies employed and the skill levels of the players involved.

Each player selects a Champion before the match begins. Champions are the playable characters of the game, each with a unique set of abilities, base statistics, and roles such as tank, assassin, mage, marksman, or support. The diversity of champions is one of the primary reasons for the game's longevity, as players must adapt their strategies based on the composition of both their own team and the enemy team.

The map itself is divided into three main lanes — Top, Mid, and Bot — connected by a forested area known as the Jungle. Defensive structures called Towers line each lane and deal significant damage to enemy units that approach them. Inhibitors, located deeper in each base, regenerate Super Minions for the opposing team when destroyed. Neutral monsters inhabiting the jungle provide buffs and gold to the players who defeat them. Throughout the game, both teams send waves of AI-controlled Minions down each lane, which automatically attack enemy units and structures.

Victory is achieved by coordinating team fights, securing objectives, and ultimately breaching the enemy base to destroy the Nexus. The following table summarizes the key technical and game-design features of League of Legends:

Feature	Description
Genre	Multiplayer Online Battle Arena (MOBA)
Players	10 (5 per team)
Teams	5 vs 5
Match Duration	20 – 50 Minutes
Platform	PC (Windows / macOS)
Networking Model	Client-Server Architecture

3. High-Level Architecture

The proposed architecture for this game is broken down into ten distinct systems, each responsible for a specific domain of functionality. This separation ensures that changes in one system do not inadvertently affect unrelated parts of the codebase. The following subsections describe the responsibilities and key classes of each system.

3.1 Player System

The Player System is responsible for managing everything related to a human player's session. This includes storing the player's profile data, tracking their in-game level and experience, handling champion selection during the pre-game lobby phase, and capturing input from the keyboard and mouse to drive champion movement and ability usage. The primary classes involved are Player, PlayerProfile, and InputHandler.

3.2 Champion System

The Champion System encapsulates all data and behavior associated with individual champion entities. Each champion has a defined health pool, mana or energy resource, base attack statistics, and a set of four abilities with individual cooldown timers. The system also manages status effects such as stuns, slows, and shields that can be applied to or removed from champions during combat. Key classes include Champion, Ability, Buff, and Debuff.

3.3 Combat System

The Combat System processes all offensive and defensive interactions between game entities. This includes resolving auto-attacks, calculating damage based on armor and magic resistance, executing skill effects, applying healing, and handling champion death and respawn mechanics. It acts as a central processor for damage-related events, delegating specific calculations to helper classes such as DamageCalculator and AttackComponent. Major classes include CombatManager, DamageCalculator, AttackComponent, and HealthComponent.

3.4 Matchmaking System

The Matchmaking System is responsible for the pre-game experience. It manages the player queue, matches players based on their skill rank, balances team composition, and creates a game lobby. Once ten players are gathered, it transitions the session to the champion select phase. Core classes include Matchmaker, QueueManager, and LobbyManager.

3.5 Networking System

The Networking System handles all real-time communication between the central game server and each connected client. It is responsible for transmitting player inputs to the server, broadcasting updated game states back to all clients, managing latency compensation, and handling disconnection recovery. Classes such as NetworkManager, PacketHandler, GameServer, and ClientConnection form the backbone of this system.

3.6 Game State Management System

This system governs which phase of the game is currently active. Using a finite state machine pattern, it transitions between states including Main Menu, Matchmaking, Champion Select, Loading Screen, In Game, Victory Screen, and Defeat Screen. Each state has well-defined entry and exit logic. The primary classes are GameManager, StateMachine, and GameState.

3.7 Map and Environment System

The Map and Environment System manages all static and semi-static elements of the game world. This includes terrain colliders, tower aggro range and attack logic, inhibitor health and respawn timers, the Nexus destruction condition, and jungle camp spawn cycles. Core classes include MapManager, Tower, Inhibitor, and Nexus.

3.8 AI System

The AI System drives the behavior of all non-player entities, primarily Minions and Jungle Monsters. Minions follow lane pathways, attack the nearest enemy, and prioritize champions that attack allied units. Jungle Monsters use patrol and aggro logic. A shared PathfindingManager handles navigation mesh queries for both types. Key classes are MinionAI, MonsterAI, and PathfindingManager.

3.9 Inventory System

The Inventory System allows players to purchase and manage items during the game. Each item provides stat bonuses such as increased health, attack damage, or ability power. The system validates gold requirements at the point of purchase and applies item effects to the champion upon equipping. The ShopManager, Item, and Inventory classes form this module.

3.10 User Interface System

The UI System renders all heads-up display elements, including champion health and resource bars, the minimap, the ability hotkey panel, item slots, the kill feed, and the post-game scoreboard. It subscribes to game events and updates the UI reactively. Primary classes include UIManager, HUDController, and ScoreboardManager.

4. Proposed Code Structure

Organizing the Unity project into a clear and logical folder structure is essential for long-term maintainability. The following hierarchy is proposed for the project:

```
Assets/
│
├── Scripts/
│   ├── Core/          → GameManager, StateMachine, GameState
│   ├── Networking/    → NetworkManager, PacketHandler, GameServer
│   ├── Player/        → Player, PlayerProfile, InputHandler
│   ├── Champions/     → Champion, Ability, Buff, Debuff
│   ├── Combat/        → CombatManager, DamageCalculator, HealthComponent
│   ├── Inventory/     → Inventory, Item, ShopManager
│   ├── AI/            → MinionAI, MonsterAI, PathfindingManager
│   ├── UI/            → UIManager, HUDController, ScoreboardManager
│   └── Managers/      → MapManager, MatchManager, AudioManager
│
├── Prefabs/           → Reusable GameObjects (champions, minions, towers)
├── Scenes/            → MainMenu, ChampionSelect, GameScene
├── ScriptableObjects/ → Champion stats, Item data, Ability configs
└── Resources/         → Dynamically loaded assets at runtime
```

The Scripts/Core folder contains the foundational classes that tie all other systems together, making it the first place any new developer should explore. Scripts/Networking is isolated so that the network layer can be swapped or upgraded independently of game logic. ScriptableObjects are used extensively for champion and item data because they allow designers to configure game parameters without touching code, improving the separation between design and engineering concerns.

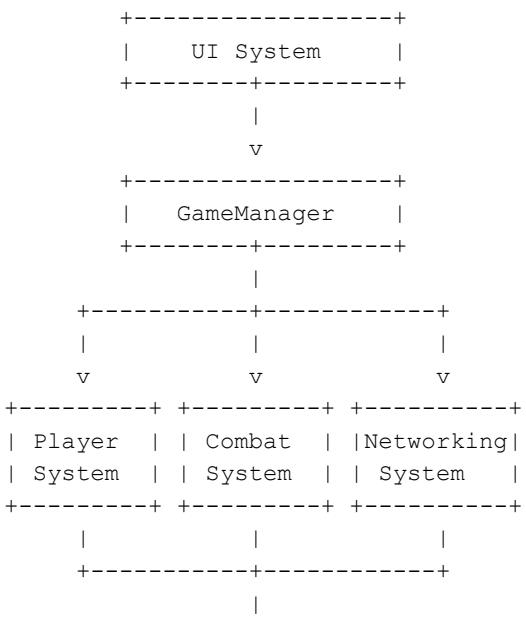
5. System Communication

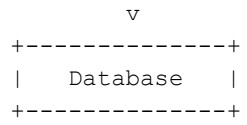
Understanding how the ten major systems interact is critical to ensuring that data flows correctly and efficiently through the game. The architecture follows an event-driven communication model complemented by direct method calls where latency is critical.

The GameManager sits at the center of the architecture and acts as the primary coordinator. The UI System subscribes to events published by GameManager to reflect real-time changes in champion health, ability cooldowns, and score. When a player performs an action, the Input Handler captures it and forwards it to the Player System, which then invokes the appropriate method in the Champion System. If the action is an attack or ability use, the Champion System delegates to the Combat System for damage resolution.

The Networking System operates in parallel with all other systems. It receives validated game actions and transmits them to the authoritative game server. The server processes these actions, updates the canonical game state, and broadcasts the result to all connected clients. The Client System then applies the received state to the local game world.

The following diagram illustrates the high-level data flow between systems:

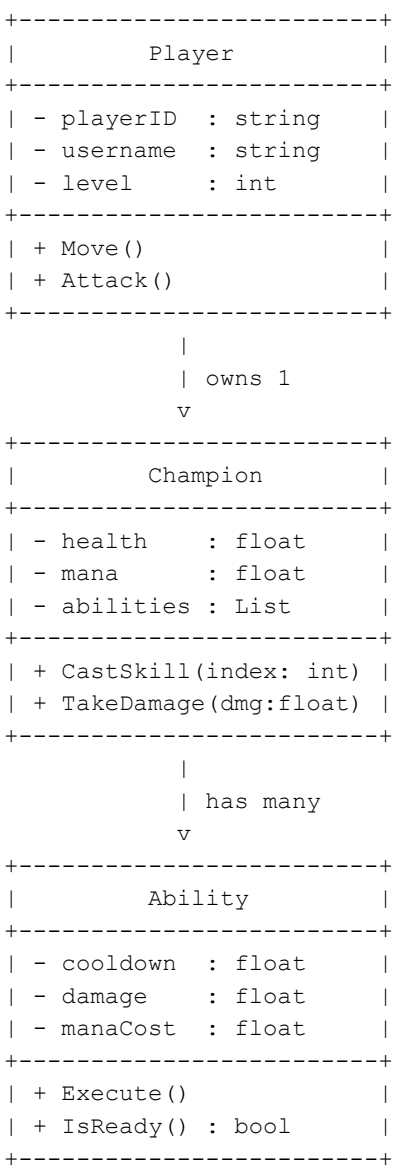




Player profile data, match history, and item statistics are persisted to a remote database after each match concludes. The database is never accessed directly by individual systems; instead, all read and write operations are funneled through the NetworkManager, which ensures that data consistency is maintained and all communications are authenticated.

6. UML Class Diagram

The following simplified UML diagram illustrates the primary inheritance and association relationships among the core gameplay classes:



The Player class represents a human participant in the match and owns exactly one Champion instance. The Champion class aggregates a collection of Ability objects, each of which maintains its own cooldown state and damage value. This composition model means that adding a new champion requires only the creation of new Ability ScriptableObjects and a new Champion prefab, without

modifying any existing class logic. The association between Player and Champion is one-to-one, while Champion and Ability maintain a one-to-many relationship.

7. Design Decisions

The architecture described in this report was selected based on several software engineering principles that are particularly relevant to large-scale multiplayer games. The following discussion justifies each major design choice.

Scalability was a primary concern from the outset. By dividing the game into independent systems, each system can be scaled or optimized in isolation. For example, if the Networking System needs to be upgraded to support a new protocol, none of the gameplay logic in the Combat or Champion systems needs to change. This decoupling is achieved through well-defined interfaces and event-driven communication patterns.

Maintainability is supported by the folder structure and the principle of single responsibility. Every class in the proposed architecture has one clearly defined purpose. The DamageCalculator does not manage inventory. The ShopManager does not handle network packets. This makes it straightforward for a developer to locate and fix a bug without risking unintended side effects elsewhere in the codebase.

Reusability is promoted through the use of Unity's ScriptableObjects for data-driven design. Champion statistics, item properties, and ability configurations are all stored as ScriptableObject assets. This means the same Ability class can represent entirely different spell behaviors simply by changing the data it references at runtime, eliminating the need for subclassing every possible ability variant.

For multiplayer synchronization, the architecture adopts a server-authoritative model. The game server is the single source of truth for all game state. Clients send input events to the server and receive validated state updates in return. This prevents cheating and ensures that all players observe the same game world, even in the presence of network latency. Client-side prediction is employed locally to reduce the appearance of lag, with corrections applied when the server state is received.

Unity's component-based architecture aligns naturally with the modular system design described here. Each champion entity in the scene is composed of multiple MonoBehaviour components such as HealthComponent, AttackComponent, and MovementController, rather than a single monolithic class. This makes it easy to enable or disable specific behaviors at runtime and to test components individually.

8. Advantages of the Proposed Architecture

The following table summarizes the primary advantages of the proposed modular architecture and provides a brief explanation of each:

Advantage	Explanation
Scalability	Each system can be extended or replaced independently. New champions, items, or game modes can be added without restructuring the core codebase.
Reusability	ScriptableObjects and component-based design allow data and behaviors to be shared across multiple game entities without code duplication.
Easy Maintenance	Clear separation of concerns means bugs are localized to specific systems, reducing the risk of cascading failures across unrelated modules.
Better Testing	Isolated systems with well-defined interfaces can be unit tested independently using mock objects, improving overall code quality and reliability.
Modularity	Systems are self-contained units. A team of developers can work on different systems simultaneously without merge conflicts or integration problems.

9. Challenges and Solutions

Developing a multiplayer game of this complexity presents a number of engineering challenges. The following table outlines the most significant obstacles and the solutions adopted in this architecture:

Challenge	Proposed Solution
Network Latency	Client-side prediction is used to simulate movement and attacks locally, while the server continuously validates and corrects the game state to keep all clients synchronized.
State Synchronization	A server-authoritative model ensures only one canonical game state exists. Delta compression is used to transmit only changed properties, reducing bandwidth consumption.
Large Number of Objects	Object pooling is implemented for frequently spawned entities such as minions and projectiles. This eliminates repeated instantiation and destruction overhead during runtime.
Memory Usage	Assets are loaded dynamically using the Resources system and unloaded when no longer needed. ScriptableObjects reduce memory duplication by sharing data instances.
Balancing Gameplay Logic	All game balance parameters, including champion stats and item values, are stored externally in ScriptableObjects. This allows designers to tune values without recompiling code.

10. Conclusion

This report has presented a comprehensive software architecture for a multiplayer game inspired by League of Legends, designed to be implemented in the Unity game engine. The analysis covered ten interconnected systems, including the Player System, Champion System, Combat System, Networking System, and AI System, each of which plays a critical role in delivering the real-time, competitive experience that defines the MOBA genre.

The importance of a well-designed architecture in multiplayer games cannot be overstated. Without a clear structural foundation, a codebase of this scale would quickly become unmanageable, leading to increased development time, more frequent bugs, and difficulty in adapting to new requirements. The modular architecture proposed here ensures that each system remains focused on a specific domain of responsibility, which in turn makes the overall project significantly easier to build, test, and maintain.

The proposed architecture supports the unique demands of a game like League of Legends in several important ways. The server-authoritative networking model guarantees a consistent game state for all ten connected players, while client-side prediction minimizes the perceived impact of network latency. The use of Unity's ScriptableObjects for champion and item data enables a clean separation between game logic and game content, allowing designers and engineers to work in parallel without interfering with each other's work.

The benefits of modular design are particularly evident when considering the long-term evolution of the game. New champions can be introduced by creating new data assets and prefabs without modifying existing champion classes. New items can be added through the ShopManager without altering the combat or inventory logic. New game modes could theoretically be supported by adding new GameState implementations to the state machine without disrupting the existing in-game flow. This level of extensibility is only achievable when the architecture is carefully planned from the beginning.

Finally, Unity's component-based development model provides an ideal foundation for the architecture described in this report. The ability to compose game entities from discrete, reusable components

aligns naturally with the object-oriented and modular design principles applied throughout. Overall, the proposed architecture is robust, scalable, and well-suited for the complexity demands of a real-time multiplayer game at the level of League of Legends.